

Mikhail LeTal — where we are and how we win

Everything that matters, as of Wednesday night

Alexander Urvancev Yan Beletskiy

Wednesday 9 September 2026, 22:45 CEST

Uploads close Friday 11 September, 11:00 London — 36 hours

In one paragraph. Our engine is uploaded, safe, and the strongest we have shipped. We lose games for one reason: it misjudges positions and walks into losses believing it is winning. Tonight we tested our biggest idea for fixing that — a neural network — and it lost by 67 Elo. Research then told us a second attempt has about a 7% chance, and pointed instead at **six changes worth roughly 100 Elo between them, every one with a measured figure from a comparable engine.** That is the plan for the final 36 hours. Winning outright is not realistic. A strong finish in Friday’s tournament is.

Contents

1	Where we stand	2
1.1	Live right now	2
1.2	How good we actually are	2
1.3	Version 1.2, the build now playing	3
2	The core problem	3
2.1	What three traced games showed	3
2.2	Why more computing power cannot fix it	3
3	The network: what happened	4
3.1	The result	4
3.2	The strangest part: four measurements all said it would win	4
3.3	Verdict on a second attempt: no, about 7%	4
3.4	Three things I had wrong, corrected	5
4	The plan that replaces it	5
4.1	Why correction history is the pick	5
4.2	The hardware finding	6

5	Timetable	6
6	Free points — do all of these	6
7	Honest assessment	7
7.1	And the thing that is genuinely ours	7

1. Where we stand

1.1 Live right now

Version 1.2 is uploaded and playing. It passed the safety gate: 500 games, 500 wins, no crashes, no illegal moves, no losses on time. That gate does not measure strength — the opponents are trivial — it measures whether the engine can lose to something other than chess. It cannot.

Rank	Rating	Record	Best streak	Checkmates
#126 of 432 (top 29%)	1800	9–6–6	2	9

That standing decides almost nothing. The rules are explicit:

“The ladder only seeds the final Swiss”

“Only the locked-build final Swiss counts, by points.”

“Tie-breaks: points, Buchholz, head-to-head, **earlier final submission.**”

Thirteen rounds on Friday afternoon decide everything. Our ladder rank affects the first pairing and nothing else. We joined late (first rated game was round 67), started low, and have therefore only ever been paired against weak opponents — **we have never played a top-50 team.**

Note the last tie-break: **submitting earlier breaks ties.** That is free.

1.2 How good we actually are

Elo comparisons across different opponents are unreliable, so we measured something better.

Average centipawn loss. For every move played, ask Stockfish how good the position was before and after. The drop is what that move threw away. Average it.
Real example from our round-85 loss: before our 13th move the position scored -24 (level). After it, -460 . That one move cost **436 centipawns** — four and a half pawns.
It ignores ratings and pairings entirely. It compares move quality directly.

	opening	middlegame	endgame
Mikhail LeTal	38.4	38.1	25.7
Emile Andrieu	7.0	10.8	7.4
THE ROOOOOKKK!!!!	10.7	11.3	20.5
Lubina	5.4	20.2	8.2

Three readings, and the third is the one that guides everything below.

Our blunder rate has nearly halved across builds — 3.1% in the old engine, 1.8% now. The work is landing.

We are worse than the top five everywhere, by roughly two to four times, and we throw away serious material about twice as often as the worst of them. That gap *is* the difference between us and them.

And the gap is widest in the opening and early middlegame, not the endgame. Our endgame is our *best* phase. Every game starts from a curated position so nobody has an opening book — everyone thinks from move one — and those early positions have every piece on the board, which is exactly where *activity* matters most and where our evaluation is blind.

1.3 Version 1.2, the build now playing

A smarter search that avoids re-examining moves it knows are bad	+35 Elo
Skipping positions already too good to need searching	+15 Elo
Three genuine bug fixes	—
Total	+50 Elo

The honest caveat: the uncertainty runs from about -2 to $+102$. The best estimate is $+50$ and the data cannot completely rule out zero. It ships on other grounds: one change is mathematically free (identical search, less work per position), and one bug fix stops the engine scoring an already-won position as a draw — a lost game, not a lost rating point.

2. The core problem

2.1 What three traced games showed

Round 85 (lost). From move 14 to move 36 — 23 consecutive moves — our engine believed it was winning by one to two pawns. It was losing by two to six. **The sign was wrong on 21 of 23 moves.** Not a blunder and not time trouble: move 13 was searched **11 moves deep over 812,000 positions** and returned “+45, I am fine” for a position scored -460 .

Round 87 (drawn). Same disease, but wrong *in both directions in one game* — too pessimistic by 100–370 centipawns for twenty moves, then too optimistic by 130–340 for the next ten. **This is the more important one:** a consistent bias could be corrected with a constant. Noise cannot.

Round 88 (lost). Traded rooks into a lost pawn endgame. At move 41 it searched **19 moves deep over 2.28 million positions** and reported -143 for a position that was -578 .

2.2 Why more computing power cannot fix it

The search is only as good as the judgement at the end of it.

The engine looks ahead, reaches positions it cannot look past, and asks its evaluation: “is this good for me?” Every decision traces back to that answer.

Our evaluation counts material and where pieces sit. It has **no concept of activity** — no measure of how many squares your pieces control. So when we are a pawn up but the opponent has two bishops raking open diagonals and open lines at our king, it reports “+200, winning.”

Searching deeper does not fix that. It explores more thoroughly *toward the wrong goal*.

This is also why a faster machine adds no strength. **The competition runs every engine on one core of the organisers’ server, 2 GB, no GPU.** We upload a zip; their machine runs it. Nothing we own is involved on Friday. A better machine lets us *test* more, which matters enormously — but through finding out what works, not through raw power.

3. The network: what happened

3.1 The result

Network vs. v1.2, 300 games at the full 2-minute control	
Result	+111 wins, 21 draws, 168 losses
Strength	−67 Elo (range −106 to −29)

Not the speed cost. The network costs 14.6% of our search speed — about a tenth of a move of lookahead, worth 5–9 Elo. The loss is 67. **So roughly 60 Elo is the network genuinely choosing worse moves.**

3.2 The strangest part: four measurements all said it would win

Check	What it measured	Winner
Held-out accuracy	Predicting Stockfish’s score	network
Round-85 positions	Knowing who was winning, in a real loss	network, 20/26 vs 5/26
Quiet positions	Accuracy where it should be strongest	network
Game outcomes	Does the side it favours actually win?	network

The last was built specifically to be trustworthy — no engine in the target, just “did this side go on to win,” over 12,000 positions from 9,200 real games. The network scored 79.0% against our hand-written 76.3%. **And it lost 67 Elo.**

3.3 Verdict on a second attempt: no, about 7%

We commissioned research into how working nets are actually trained. The literature is broadly *favourable* to trying again — and the answer is still no, for a specific reason.

The one real deviation is data. The working range across the field is **50–3000 training positions per parameter. We had 1.9.** The lowest volume anyone reports a working net at is 1.5 million positions; we had 186,000. A master’s thesis abandoned precisely our approach — online games labelled by an engine — as “too small for this task.”

But that is not what cost us 67 Elo. A starved network is one that judges positions badly. Ours judged them *better* than our hand-written evaluation on four independent measurements, including at real search leaves, and still played 67 Elo worse.

And the three mechanisms proposed as the fix have each already been tested here and killed: the outputs are not compressed (they are 5–19% *wider*); the error does not explode on unusual positions (the network stays ahead there); and the improved training target is one **the network already wins.**

So the honest position is not “we know what went wrong and can fix it.” It is **“the network was better by every measure we own and played worse, and nobody has explained why.”** The right response to an unexplained result with 36 hours left is not to rebuild it ten times larger.

0.7 (pipeline correct first time, no iteration) $\times 0.35$ (true effect positive at all) $\times 0.3$ (an affordable test resolves it) $\approx$ **7%.**

3.4 Three things I had wrong, corrected

1. “The architecture cannot see kings, and no training fixes that.” I called this fatal. **768→128 is what other engines beat their hand-written evaluations with** — one reports **+200 Elo** from it; another measured a width curve where **128 is the peak** and 512 is worse. The standard training tool’s author advises starting exactly there. I invented a fatal flaw that is not one.
2. “We have an incremental accumulator.” We do not. The code rebuilds it from scratch on every call, deliberately, and says so in its own comment. I stated the opposite repeatedly.
3. **Mobility as the leading candidate. Ruled out.** Worth $+41 \pm 14$ elsewhere, but at our measured $1.7\times$ time-to-depth it costs **60–65 Elo of lost search — a net loss**. Other engines pay almost nothing because they count attacks their move generator already produces; ours needs 4–6 hours of optimisation before the term is even affordable.

4. The plan that replaces it

Every figure is measured in a comparable engine, with its uncertainty.

Change	Elo	Hours	Note
SEE	+26	0	Being tested right now , result 06:20. Makes the search <i>faster</i> , and targets our blunder rate at its likeliest source.
Internal iterative reduction	$+9.7 \pm 5.5$	0.3	Three lines. Best Elo per line available, and cannot interact badly with anything.
Transposition table 21 → 23	untested	0.1	Direction certain. See below.
History gravity + malus	$+15.9 \pm 7.4$	1	~15 lines, no measurable cost.
Correction history	$+18.9 \pm 8.0$	3	The one aimed at what we measured.
Time management	+28 to +43	2	Real, but two constants must move together or we risk losing on time — a whole point in a Swiss.

4.1 Why correction history is the pick

We measured that at *real search leaves* our evaluation is off by a median of **304–342 centipawns** — about a knight — while our pruning thresholds are 150–750. **We are making decisions on differences smaller than our own error.**

Correction history records, for each material configuration, **how far the search’s verdict usually lands from the static evaluation**, and adds that learned offset back before those

decisions are taken. It is the only item that attacks the pathology we actually observed rather than one we assumed.

One array lookup. It passed 128,672 games of testing in Stockfish, and it *scales up* with longer time controls — ours is the long one.

4.2 The hardware finding

Our machine	16 threads, 7 GB RAM, 1 GB available
Transposition table	2,097,152 slots = 59 MB
Positions searched in one move	1.6–3.2 million
Platform allows per process	2 GB

Our memory of past positions is smaller than a single move’s search. Raising it one step is one line — but testing it needs $235 \text{ MB} \times 24 \text{ processes} = 5.6 \text{ GB}$, on a machine with 1 GB free. **It has never been tested because our machine physically cannot test it.**

That is the real case for renting a second box, and it is stronger than the network case was: **a second machine roughly doubles the number of measured decisions we can make before Friday**, and the number of measured decisions is the binding constraint on this whole project.

Specification: 16+ dedicated vCPU, 32 GB RAM, Ubuntu 24.04, hourly billing. Roughly \$15–40 for the whole window. **No GPU** — our network trains in minutes on a CPU and cannot run on a GPU during a game.

5. Timetable

Thu 06:20	SEE result lands	decides the base build
Thu 06:20–10:00	Code: table size, iterative reduction, history malus	1.4 h
Thu, in parallel	Correction history	3 h
Thu 10:00–22:00	Two tests per machine, two machines	~12 h
Thu 22:00	Freeze , 500-game safety gate	1 h
Fri 08:00	Upload — early submission is a tie-break	
Fri 11:00	Locked	
Fri afternoon	13-round Swiss	

Three hours of margin. The network plan fitted the same window with *zero* slack, on a throughput figure nobody had measured, and would have consumed the second machine entirely.

6. Free points — do all of these

1. **Upload the final build early Friday, not at 11:00.** Explicit tie-break; ties on points are common over 13 rounds.
2. **Never lose to a crash, an illegal move, or the clock.** Each is a whole point given away. The 500-game gate exists for this and must run on whatever ships.
3. **Keep uploading until the lock.** Ten a day, latest valid one plays, a failed validation costs nothing. Every rated game is a free reliability test.

4. **Fix the time allocation.** In round 87 the engine spent **1.0 second on the losing move while holding 88 seconds**, and 9.4 seconds on a move in an already-lost endgame, finishing with 32.9 unused. Spending time inversely to where it matters is a defect.

7. Honest assessment

Winning outright is not realistic. The top of the field is measurably better at chess — a blunder rate of 0.0–0.9% against our 1.8% — and closing that is an evaluation rebuild, not a day of tuning.

A strong Swiss finish is realistic and is the right target. Thirteen rounds on points, locked builds, ladder seeding nearly irrelevant. Consistency matters more there than peak strength, and consistency is what a lower blunder rate buys. Ours has already halved once.

What would have to be true for a top-five finish:

1. the test running tonight lands positive (expected +15 to +25);
2. three or four of the six changes land and stack;
3. we give away nothing to crashes, flags or lost time — entirely in our control;
4. the Friday pairings are not brutal early — not in our control.

7.1 And the thing that is genuinely ours

Every number here has a matched comparison, a stated uncertainty, and a caveat where one is owed. **Four times today a persuasive measurement said we were winning and the games said otherwise** — and each time it was caught, because someone wrote down in advance what would count as evidence.

Five separate defects today shared one shape: a check that could not fail. A test set defined using the training data. A “wait until the machine is free” loop whose own command matched its own search. A data filter that called the function under test. A guard written to catch the incident that had already happened. A benchmark that verified how fast an answer arrived but never that it was the same answer.

The question that catches all five, now in our decision log:

What is this check defined in terms of, and is that the thing I am testing?

That is what we can defend in front of a judge, and it is worth more than any single technique.